Deep learning Cardiac Cine MRI Segmentation

Ziheng Wang[a], Yuhao Zhang[a], and Peixuan Zhu[b]

[a]School of Information Science and Technology
[b]School of Life Science and Technology

April 15, 2023

Abstract

In this project, we investigated the performance of deep neural network in cardiac cine MRI segmentation by implementing several deep network, including UNet, UNet without skip connection, and UNet++, which is a UNet plus extra upsampling blocks. Our results show the significant improvement of the skip connection in UNet and the substitution of UNet++ to UNet, in both segment performance and model size.

1 Data Preprocessing

To enhance visualization and facilitate alignment between network predictions and ground truth, as well as to accurately calculate the dice coefficient, we initially partitioned the dataset into three distinct channels, which correspond to the right ventricle (RV), myocardium (MYO), and left ventricle (LV), respectively.

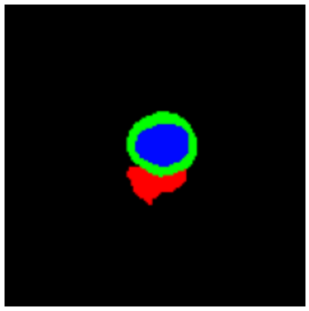


Figure 1: Three classes segmentation

The process should follow these steps:

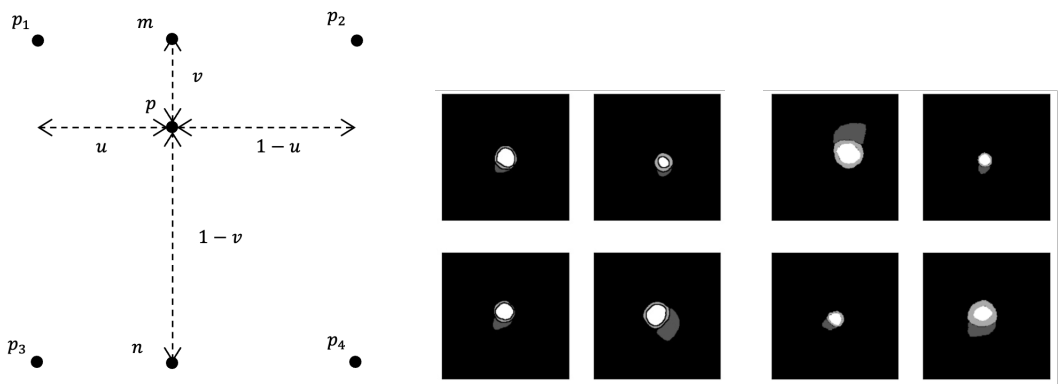
1. Identify pixels with values of 85, 170, and 255, respectively.
2. Display the pixels in three separate images that are not aligned with each other.
3. Combine the images along the channel dimension to synthesize a three-dimensional image.

However, the bilinear interpolation, which is set as the default mode of resize operation in the data loading process, has a disadvantage dealing with the sudden pixel value change between two different areas.

Detailedly, in bilinear interpolation, a target pixel p is derived by doing linear interpolation twice to the nearest four pixels p_1, p_2, p_3, p_4 in the original image.

$$\begin{cases} m = (1-u) \cdot p_1 + u \cdot p_2 \\ n = (1-u) \cdot p_3 + u \cdot p_4 \\ p = (1-v) \cdot m + v \cdot n \end{cases}$$

For example, if $u = v = 0.4$, $p_1 = p_2 = p_3 = 85$, $p_4 = 170$, then $p = 106.25$, which is not included in 85, 170, 255, leading to a gap in the synthesized image (See Figure 2(b)). Therefore, we switch to the NEAREST mode of interpolation and set thresholds, the gap then is eliminated and the results are much better compared to bilinear interpolation.



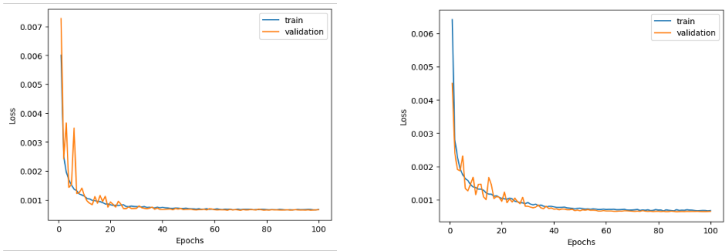
(a) Bilinear interpolation (b) Gap using bilinear interpolation (c) Bilinear interpolation

Figure 2: Data enhancement

2 Using Different Network Structures

2.1 UNets

We evaluated the performance of UNet architectures with and without skip connections. The model with skip connections demonstrated superior performance compared to the model without them. Specifically, with skip connections, the network was observed to converge at approximately 40 epochs, while the model without skip connections required around 50 epochs to converge. Furthermore, the Dice coefficient, a measure of segmentation accuracy, was significantly better in the model with skip connections. These results suggest that the incorporation of skip connections can significantly enhance the performance of UNet models for segmentation tasks.



(a) With skip connection (b) Without skip connection

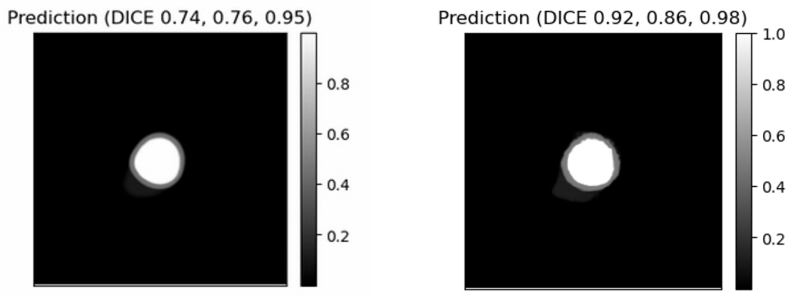
Figure 3: Loss curve comparison

	mean_RV	*mean_MYO*	*mean_LV*	*std_RV*	*std_MYO*	*std_LV*
with shortcut	**0.838**	**0.858**	**0.947**	**0.103**	**0.039**	**0.023**
no shortcut	0.723	0.772	0.911	0.127	0.045	0.024

Table 1: Performance Comparison

In addition, we observed a pronounced difference between the segmentation outputs of UNet models with and without skip connections. Specifically, the model without skip connections produced a highly smooth segmented image, indicating a preference for low-frequency components in the input image.

Contrary to this, the model with skip connections generated a more rough segmentation output that more closely approximates the ground truth. This discrepancy may be attributed to the information loss that occurs during the downsampling stage of the network, due to the incomplete representation of the transform domain.



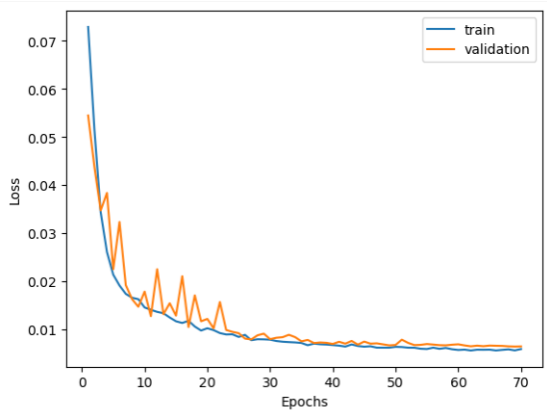
(a) With skip connection (b) Without skip connection

Figure 4: Segment performance comparison

2.2 Soft-dice Loss

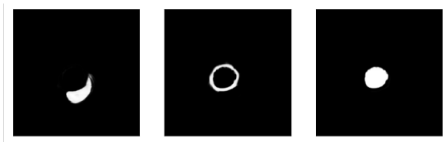
Following the adoption of the soft-dice loss function, a significant decline in model performance was observed. Specifically, the loss curve exhibited slow convergence, requiring approximately 70 epochs, in contrast to 40 epochs for UNet models with skip connections and 50 epochs for models without skip connections, when using binary cross-entropy loss.

Additionally, the evaluation of the Dice coefficient on RV, MYO, and LV channels demonstrated a continued decline in performance, with a maximum value occurring at approximately 220 epochs. These results suggest that the choice of loss function has a significant impact on the performance of the segmentation model.



(a) Loss curve

(b) epoch=80



(c) epoch=160

Figure 5: Performance with soft-dice loss

	mean_RV	*mean_MYO*	*mean_LV*	*std_RV*	*std_MYO*	*std_LV*
BCE, epoch=60	**0.838**	**0.858**	0.947	**0.103**	**0.039**	**0.023**
DICE, epoch=220	0.824	0.852	**0.948**	0.108	0.042	0.026

Table 2: Performance Comparison

A possible reason for that is their different loss characteristics: Dice loss is a region-based loss function that measures the overlap between the predicted and ground truth segmentation, while Binary Cross Entropy loss is a pixel-wise loss function that measures the difference between individual pixel predictions and the ground truth. The characteristics of these two loss functions are different, which may lead to different convergence rates and performance.

2.3 UNet++

2.3.1 Introduction

To deal with the uncertain number of levels providing the optimal performance problem in UNet, UNet++ fills the inside of UNet with several upsampling blocks and skip connections, enabling it to better capture both local and global contextual information. Moreover, the dense skip connections facilitate more effective gradient flow during training, leading to better convergence and performance.

Yet the dense structure makes it contain more parameters than UNet, one solution as well as a benefit of the structure is its prunable property. Levels of UNet++ are dependent on others in training stage, however independent in testing. Pruning the deeper level of the model before testing has proved to be a way to reduce the size of it while preserving most performance.

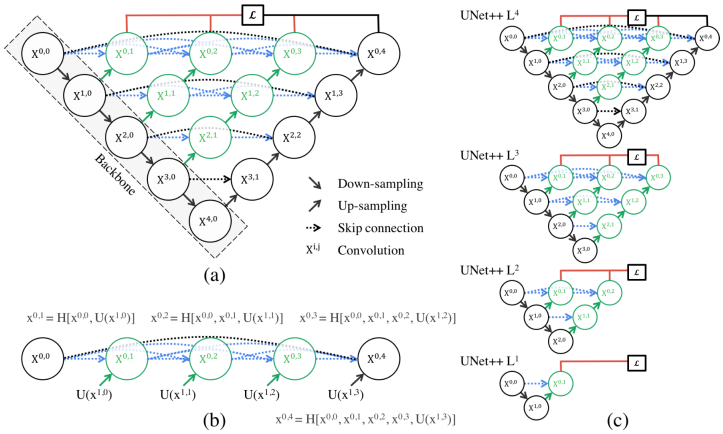
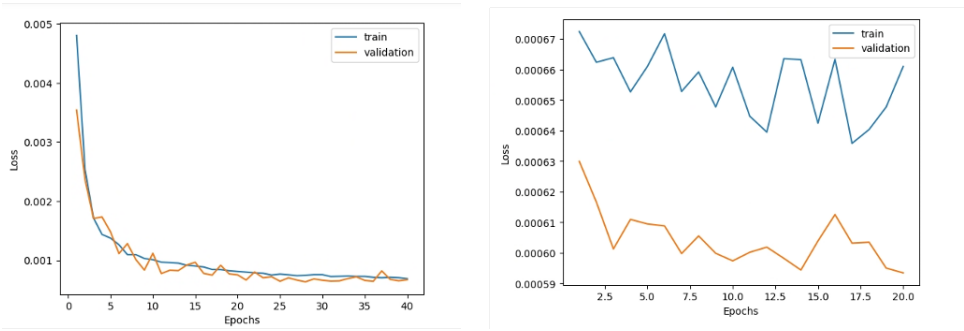


Figure 6: UNet++ structure

2.3.2 Testing

We then test the convergence and performance of UNet++. The results are shown below.



(a) epoch=0-40

(b) epoch=60-80

Figure 7: Loss curve

	mean_RV	*mean_MYO*	*mean_LV*	*std_RV*	*std_MYO*	*std_LV*
UNet	0.838	0.858	0.947	0.103	0.039	0.023
Unet++	**0.840**	**0.874**	**0.956**	**0.095**	**0.031**	**0.019**

Table 3: Performance Comparison

It proved to be a lot of improvement than UNet, while the size of UNet is around 120,000 KB, UNet++'s size is around 46,000 KB, which is 3 times smaller than UNet.

As mentioned above, the prunable property is valueable for UNet++. Since we firstly train the 3 level UNet++, we reduce it to 2 level, and one level, respectively. We witness only a little decrease after we switch to 2 level UNet++ (10000 KB), but a more reduce of performance when switching to 1 level model (2000 KB). However, the 1 level one still gives better result than UNet without skip connection.

	mean_RV	*mean_MYO*	*mean_LV*	*std_RV*	*std_MYO*	*std_LV*
UNet	0.838	0.858	0.947	0.103	0.039	0.023
UNet no shortcut	0.723	0.772	0.911	0.127	0.045	0.024
UNet++ L^1	0.782	0.823	0.925	0.137	0.047	0.023
UNet++ L^2	0.826	0.849	0.940	0.100	0.039	0.026
Unet++ L^3	**0.840**	**0.874**	**0.956**	**0.095**	**0.031**	**0.019**

Table 4: Performance Comparison